4.3.1. Kiểm thử câu lệnh và Độ bao phủ câu lệnh (Statement Testing & Statement Coverage)

📋 Khái niệm cơ bản

Trong **kiểm thử câu lệnh** (statement testing), các **mục tiêu bao phủ** (coverage items) là các **câu lệnh có thể thực thi** (executable statements). Mục tiêu là thiết kế các test case sao cho chúng thực thi các câu lệnh trong mã nguồn cho đến khi đạt được mức độ bao phủ chấp nhận được.

Ví dụ minh họa:

```
def calculate_discount(price, customer_type):
    discount = 0                            # Câu lệnh 1
    if customer_type == "VIP":             # Câu lệnh 2
        discount = price * 0.2             # Câu lệnh 3
    else:                                  # Câu lệnh 4
        discount = price * 0.1             # Câu lệnh 5
    return price - discount                # Câu lệnh 6
```

📊 Cách đo lường độ bao phủ

Công thức:

```
Độ bao phủ câu lệnh = (Số câu lệnh đã thực thi / Tổng số câu lệnh có thể thực thi) × 100%
```

Ví dụ: Với hàm trên (6 câu lệnh), nếu test case chỉ chạy với `customer_type = "VIP"` :

- Câu lệnh được thực thi: 1, 2, 3, 5 (4 câu lệnh)
- Độ bao phủ = 4/5 × 100% = **80%**

✅ Giá trị của độ bao phủ 100%

Khi đạt 100% statement coverage:

- Tất cả câu lệnh có thể thực thi đã được chạy ít nhất một lần
- Mỗi câu lệnh chứa **lỗi** (defect) sẽ được thực thi → có thể gây ra **thất bại** (failure) để phát hiện lỗi đó

⚠️ Các hạn chế quan trọng

Hạn chế	Giải thích	Ví dụ
Không phát hiện hết lỗi phụ thuộc dữ liệu	Thực thi câu lệnh không đảm bảo test với mọi giá trị đầu vào	Lỗi chia cho 0 chỉ xảy ra khi mẫu số = 0: `result = a / b`
Không đảm bảo test hết logic quyết định	Có thể bỏ sót các nhánh (branches) trong điều kiện	`if (a > 0 AND b > 0)` - có thể chỉ test trường hợp cả 2 đúng
Không test đủ các đường đi	Chỉ đảm bảo mỗi câu lệnh chạy 1 lần, không đảm bảo test mọi tổ hợp	Có thể bỏ sót các kịch bản phức tạp

Ví dụ cụ thể về hạn chế:

```
def process_order(quantity, in_stock):
    if quantity > 0 and in_stock:  # Điều kiện có 2 nhánh logic
        status = "approved"         # Câu lệnh A
    else:
        status = "rejected"         # Câu lệnh B
    return status
```

Test với `quantity=5, in_stock=True` → đạt 50% statement coverage (chỉ chạy câu lệnh A) Nhưng chưa test trường hợp: `quantity=0, in_stock=True` hoặc `quantity=5, in_stock=False`

💡 Lưu ý quan trọng

Độ bao phủ nhánh (branch coverage) **bao hàm** độ bao phủ câu lệnh:

- 100% branch coverage → tự động đạt 100% statement coverage ✓
- 100% statement coverage ≠ 100% branch coverage ✗

Kết luận: Statement coverage là mức độ bao phủ cơ bản nhất, cần kết hợp với các kỹ thuật khác (như branch coverage) để đảm bảo chất lượng kiểm thử tốt hơn.

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
    - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
    - Follow Fanpage: https://www.facebook.com/hoctest/
    - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
    - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
    - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.